

1. PROJECT OVERVIEW

- **Project Title:** ComicCraft - AI Comic Story Creator using Gemini Models

2. INTRODUCTION:

This document outlines the core Python implementation used to orchestrate the AI pipeline. Furthermore, it details the practical troubleshooting solutions engineered to resolve local environment restrictions and unpredictable external network failures during development.

3. CORE PIPELINE IMPLEMENTATION

A. Application Controller (app_routes.py) The central FastAPI logic operates asynchronously to prevent server freezing while waiting for external cloud responses:

Python

```
@router.post("/generate", response_class=HTMLResponse)
async def execute_comic_pipeline(request: Request,
story_prompt: str = Form(...), character: str = Form(...)):
    # 1. Narrative Architecture
    full_query = f"Write a comic regarding {story_prompt}.
    Hero: {character}."
    scene_outline = fetch_outline_from_gemini(full_query)
    comic_script = fetch_script_from_gemini(scene_outline)

    # 2. Visual Synthesis Loop
    generated_assets = []
    for idx, panel_data in enumerate(scene_outline):
        img_location =
    synthesize_image(panel_data["image_prompt"],
    f"scene_{idx}.jpg")
        generated_assets.append(img_location)
```

```

# 3. Compilation & Web Render
document_layout = build_page_layout(generated_assets,
comic_script, scene_outline)
final_pdf = construct_pdf(document_layout)
return templates.TemplateResponse("preview.html",
{"request": request, "layout": document_layout, "pdf":
final_pdf})

```

B. Resilient Visual Processing (image_synthesis.py) A primary developmental challenge was handling Hugging Face API rate limits. The following fallback logic ensures the application never crashes:

Python

```

def synthesize_image(prompt_text,
file_name="static/panels/default.jpg"):
    try:
        api_key = os.getenv("HF_API_KEY")
        # Standard HTTPS Request logic via urllib...
        return file_name
    except Exception as network_error:
        print(f"API Timeout. Triggering Local Fallback:
{network_error}")
        # Local PIL Fallback Generation
        emergency_img = Image.new('RGB', (512, 512),
color="#E0E0E0")
        drawer = ImageDraw.Draw(emergency_img)
        drawer.text((40, 40), "IMAGE SYNTHESIS
UNAVAILABLE\nPLEASE PROCEED", fill="red")
        emergency_img.save(file_name)
        return file_name

```

4. DEVELOPMENT TROUBLESHOOTING LOG

- **Obstacle 1: Virtual Environment Execution Block**

- *Error:* PowerShell blocked the activation of `env\Scripts\activate` due to system security settings.
- *Solution:* Modified the local permissions by running `Set-ExecutionPolicy RemoteSigned -Scope CurrentUser`, allowing script execution.

- **Obstacle 2: Directory Path Length Restrictions**

- *Error:* Large Python packages (like PyTorch dependencies for diffusers) triggered a `[WinError 206]` during `pip install`.
- *Solution:* Overrode the legacy Windows 260-character path limit via the Registry Editor (`LongPathsEnabled`).

- **Obstacle 3: Module Unavailability During Boot**

- *Error:* The Uvicorn server crashed instantly with a `ModuleNotFoundError` for Jinja2 and FastAPI.
- *Solution:* Identified a mismatch in the Python PATH. Reactivated the virtual environment and re-ran `pip install -r requirements.txt` to sync dependencies correctly.

- **Obstacle 4: ISP Network Resolution Failures**

- *Error:* Specific ISPs blocked direct calls to Hugging Face, returning a `getaddrinfo failed TCP error`.
- *Solution:* Developed the autonomous Python PIL image generator (shown in the code snippet above) to intercept the crash and generate temporary graphic placeholders locally, allowing the PDF compiler to finish successfully.